Promo Finder: ranked promos from a microdrama screenplay

Problem 1 (Find the Promos). The tool reads the screenplays of a show and returns ranked 20 to 60 second promos. For each promo it gives the episode and the start and end of every segment, the hook, the cliffhanger, one line on why it works, and a rank. It can be run again on a new show with any OpenAI-compatible model.

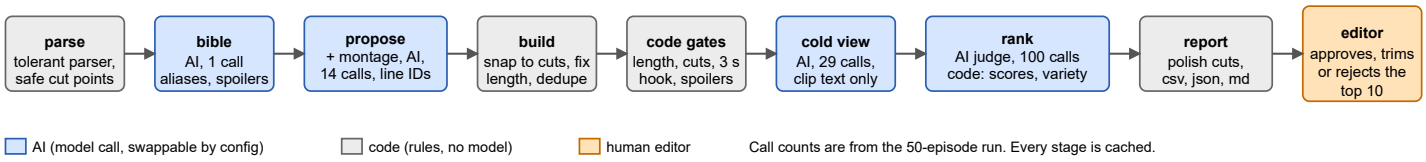
Why Problem 1

An editor's work grows with every new show: they watch every episode and pick moments by hand, and Kuku releases new shows every week. The output can also be checked with clear rules: a clip must be 20 to 60 seconds long, have a hook in the first 3 seconds, start and end on clean cuts, and end unresolved. The input pack gives 50 real episodes to test on, and the assignment asks for a tool that can be run again on a new show.

Assumptions

- I judge promos from the **script text only**: no video, audio or picture. "Makes sense on its own" means that a reader who has never seen the show can say who the people are, what is at stake and what happens next.
- **Episodes 1 to 10 are treated as the free episodes**, so a hook from them gets a small bonus (+0.05, setting `free_eps`). The show is in English for a US audience. A promo is one stretch or up to three segments, and a cut never falls inside a spoken line.
- **The promo shape** follows public trade sources (vendor blogs, not platform documents): a conflict or shock in the first 1 to 3 seconds, and a cliffhanger that does not answer its own question. Most DramaBox US ads run about 46 to 64 seconds (admapix.com, genra.ai). The number of free episodes varies from 8 to 18 across apps, so 10 is an assumption.
- Where the brief said nothing, I chose the simplest option and wrote it down in `docs/assumptions.md`. Clips are sent to Gemini and Groq. Free tiers may use the data for training, so a confidential show should use a paid or private endpoint.
- There are no hand-labelled answers, so how closely the tool agrees with an editor is **not measured**.

How it works: AI, code, and where the editor approves



The models only return **line IDs**. Code turns the IDs into seconds and enforces the rules: 20 to 60 seconds, a hook within 3 seconds of the start, cuts that fall inside no dialogue, and segments in order. The cold view (a model that sees only the clip) and the judge see only the clip text, with no episode numbers and no scores. The judge answers three separate questions (hook, ending, clarity), each in both orders, to cancel position bias. Each model is a setting in a YAML file, with limits, fallbacks and a response cache. `--profile-map` swaps a model at run time, and an offline keyword mode needs no API. The editor stays in the loop: a Streamlit review app shows the ranked list with transcripts, and the editor approves, trims or rejects each promo.

Cost and scale

Measured on the 50-episode pack (103 minutes): **147 model calls, 426k tokens (390k in, 36k out), about 12 minutes, $0** on free tiers. At paid Gemini prices (ai.google.dev/gemini-api/docs/pricing, checked 9 Nov 2026) one show costs about **$0.21** with every step on Gemini 3.5 Flash-Lite ($0.30 in, $2.50 out per 1M tokens). It costs about **$0.32** with Gemini 3.6 Flash on the three long-context steps (promotional price $0.75 in, $3.75 out, until 31 Dec 2026), and about **$0.52** at its list price ($1.50 in, $7.50 out). Multiply by the number of shows: 10 shows a week cost about $2 to $5 a week, and 50 shows about $10 to $26. (I do not know the real weekly count.) On free tiers the limit is requests (5 to 15 per minute and 20 to 500 per day per model), not tokens, and a paid key removes it. Changing one prompt reruns only the stages after it. The editor reviews about 10 clips per show instead of watching 50 episodes. For a 200-episode show, `promo estimate` predicts about 540 calls and 1.6M tokens, so the default limit of 400 calls in `pipeline.yaml` has to be raised. The parser is tested on 7 format variants (for example Windows line endings, lower-case headers and a missing scene end).

How I know it is good, and where it is not

I compared the tool's top 10 with four baselines. For each baseline, an LLM judge compared the clips pair by pair, once in each order. A win counts 1 and a split decision counts 0.5. There are 10 pairs per baseline, so one pair moves a number by 0.1. I ran this with two judge models, on the top 10 as it stood before the last polish change (one clip differs). The files are `eval_gemini_judge.json` and `eval_groq_judge.json`.

Baseline	Tool win, Gemini judge	Tool win, Groq judge
random 45 s windows	0.85	0.85
last 45 s of an episode	0.75	0.75
offline keyword heuristic	0.70	0.55
one prompt over the whole show	0.65	0.40

Perturbation: original preferred	early	late
gemini-3.5-flash-lite (structured)	0.35	0.36
gpt-oss-120b (structured)	0.25	0.44
gemini-3.5-flash-lite (first judge)	0.35	0.44
gpt-oss-120b, after the first polish	0.30	0.44

The tool clearly beats random windows and the last 45 seconds, but not the one-prompt baseline. It wins 0.65 with one judge and 0.40 with the other, so I cannot claim to beat it. **The judge also fails my test of it.** I gave it a clip and a copy with the hook

buried 8 seconds earlier (early), or with the answer shown (late). It should prefer the original more than 80% of the time, but it prefers it only 25 to 44% of the time, with both models and both prompts (30% and 44% after the first polish). So the judge's ranking cannot be trusted to punish a buried hook or a late cut, and the win rates show agreement with an LLM, not with an editor. Position bias fell from 39.6% to 20.8% of pairs after I split the judge into three questions. Limits: no hand labels, text only, 10 pairs per row.

A second check needs no judge model. A rule-based defect audit (`promo audit`) runs the same code checks on every clip set and needs no labels. A clip has a defect if it opens mid-sentence, has no speech in the first 3 seconds, ends mid-sentence, or contains the answer to an open question or a spoiler line (`audit.md`).

Clip set	Clips	Any defect	Clip set	Clips	Any defect
tool top 10	10	0.10	random 45 s windows	50	0.26
tool top 20	20	0.05	last 45 s of each episode	50	0.22
keyword heuristic top 10	10	0.00	one prompt over the show	10	0.20

The tool has fewer defects than random and last-45 windows. The keyword heuristic has none, so this check cannot tell the two apart: it finds broken cuts, not weak drama. A second, independent model run finds only 0.30 of the tool's top 20, so the list depends on the model and should be read as candidates for an editor.

Where it picks badly (`docs/failures.md`; found with code checks and the tool's own ratings, not by watching the clips):

- **Answer inside the clip:** 4 promos contained a line that the bible (the story summary) marks as the answer to an open question, either their own or another one, which spoils it. The cold view still called them open-ended. A final code step cuts the end before that line (2 fixed) or marks the promo G8 and moves it to the bottom (2, one was a rank 3 montage). It knows only the bible's 5 open questions, so other early answers are not caught.
- **Spoilers:** 7 promos show a line that the bible marks as a twist. The bible lists only 7 such lines for 50 episodes, and this check is only a soft penalty.
- **Near-duplicates:** some pairs just under the build thresholds (0.5 time overlap, 0.6 word overlap) got through. The final step uses 0.4 and 0.5 and dropped 2. These numbers are a judgement call.
- **Mid-sentence starts:** 3 of the top 30 opened on the tail of a sentence. The final step moved 2 to open on the hook line. The third (rank 6) is not fixed, because its hook line itself starts mid-sentence.
- **Weak hooks and no payoff:** 4 promos have a hook rating of 2, and 25 of 71 have no link to a question that the show later answers (ranks 4, 8, 13, 14 and 15 are in the top 15).
- **Gaps:** the model proposed nothing for episodes 6 and 8, and 3 of the top 10 end exactly where an episode ends (the show's own built-in cliffhanger).

AI tools used

I used **Claude Code** (Claude Sonnet 5.5) to help write the code and tests from a written spec. The tool itself calls **Gemini 3.6 Flash, Gemini 3 Flash, Gemini 3.5 Flash-Lite and 3.1 Flash-Lite** (free tier) and **Groq gpt-oss-120b**. **One place the AI got it wrong:** the assistant reported that the tool beat every baseline with a 0.60 win rate against the one-prompt baseline, but `eval.json` said 0.4444. The baseline is itself a model reply, and an invalid first reply is never cached, so the second eval run asked again and got 9 clips instead of 10. The mismatch was caught when the numbers were checked against `eval.json`. The baseline is now saved and reused, and every number here is read from the eval files. Other misses: a model that never answered (about 19 minutes lost to retries), and a first proposal prompt that gave 97-second spans.

Conclusion

Promo Finder turns 103 minutes of watching into a ranked list of about 10 clips for an editor to review, at roughly $0.20 to $0.50 per show, and every clip follows clear rules for length, hook and cuts. It cannot yet show that its picks beat an editor's. The next step is to have an editor label a few episodes and compare.

Links

Code (GitHub)	https://github.com/PriyanshiJ-web/kuku-promo-finder
Live demo (invented show)	https://kuku-promo-finder-xzranjxdxkmuh8app7qprg6.streamlit.app/
Raw outputs (Google Drive)	Files in CSV, Markdown and JSON format. Start with README.txt, promos.csv and report.md. JSON cannot be previewed in Drive, so download it.
Walkthrough video (Google Drive)	https://drive.google.com/file/d/1nGha9NSeU4AZkEOr0B4QIDKe1exsr7F-/view?usp=sharing

Priyanshi Jain | Roll number 220823 | B.Tech, Aerospace Engineering